



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Parallelism II: Data-Level Parallelism, SIMD



Run Demo

Download `lec31_code.zip` from Lecture Drive

Then, on local:

```
$ scp lec31_code.zip hiveNN:~/.
```

On hive:

```
$ unzip lec31_code.zip
```

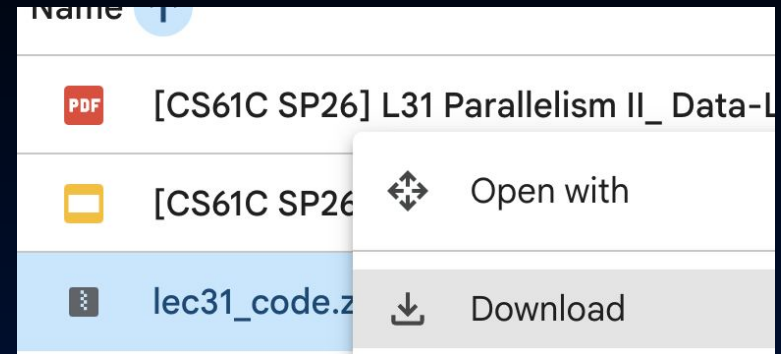
```
$ cd lec31_code/
```

```
$ make
```

```
$ ./compareruntimes.sh 512
```

Check individual files

	4/5	
Fri 4/10	Lecture 31: Parallelism II - SIMD	
	Slides Drive Video	



Compare `dgemm` functions. Which one is fastest? Why?





Agenda

dgemm, continued

Double(-Precision
floating-point) GEneral
Matrix Multiplication

- Dgemm, continued
- Flynn's Taxonomy
SIMD and Data-Level
Parallelism
- SIMD dgem
Intel Intrinsics



Summary / In practice, use GCC

Can we go faster...?

- Python vs. C
- int vs. float
- Registers
- Cache blocking
 - Transpose
 - Submatrix multiplication
- [lab] Loop unrolling

Compare dgemm functions. Which one is fastest? Why?

vim tips

- open matmul.c
- Split and arrange views to compare dgemm functions

vim cheatsheet

- `:vsplit` vertical split view
- `:split` horizontal split view
- `<ctrl-w>+<arrow>` change view
- `:q` close view
- `<ctrl-z>` pause vim temporarily and go to bash
- `fg` from bash, bring vim back foreground





In practice: GCC + good memory access design

GCC options:

<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

gcc -O2

gcc -O3

gcc handles **most** micro optimizations at the program level!

What are some differences between -O2 and -O3?

```
$ make run_matmul_02
```

```
$ ./run_matmul_02 0 mats/testA_512.mat mats/testB_512.mat
```

```
$ # do same for run_matmul_03
```





In practice: GCC + good memory access design

GCC options:

<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>

gcc -O2

gcc -O3

gcc handles **most** micro optimizations at the program level!

GCC can only optimize based on the provided high-level **program**.

- It does not know how to change the "algorithm":
 - Transpose matrices/create submatrices
 - Multithread/use parallelism
- Programmer task still involves writing good code:
 - Minimizing **compulsory cache misses**
 - **Parallelizing** where possible



Two Observations, Made in Parallel

- Element-wise multiplication (of row of A, column of B) can be computed independently
 - **One FP instruction:** element-wise multiplication
- Different elements of matrix C can be computed independently
 - **Separate sequences** of `fmul` + `fadd`

Perhaps...parallelism?



Flynn's Taxonomy

- Dgemm, continued
- Flynn's Taxonomy
SIMD and Data-Level Parallelism
- SIMD dgem
Intel Intrinsics



Why Parallelism Processing?

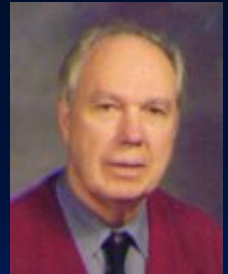
- CPU clock rates are no longer increasing.
 - Technical/economic challenges: advanced cooling technology too expensive/impractical for most applications; energy costs prohibitive
- Parallel processing is the only path left to better performance.
 - Airline analogy: Max air speed is limited by economics, so use more + larger airplanes to increase throughput (...and smaller seats)
- We will focus on parallel computing in this module:
How can we make **one program/task** run faster?

Flynn's Taxonomy

- **Flynn's Taxonomy** is a classification system of parallel hardware proposed in 1966 and still used today.

		Data Streams	
		Single	Multiple
Instruction Streams	Single	SISD	SIMD
	Multiple	MISD	MIMD

P&H Figure 6.2



Prof. Michael
Flynn
Stanford

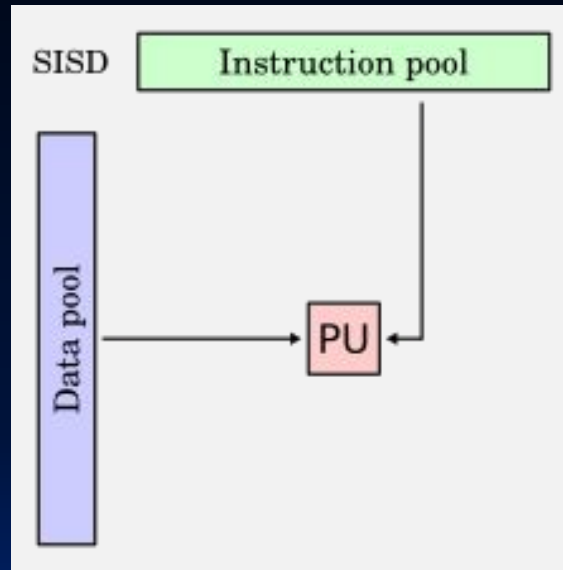
Yan, SP26

SIMD, MIMD are the most commonly encountered today—usually both in the same system!



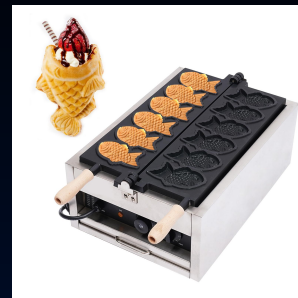
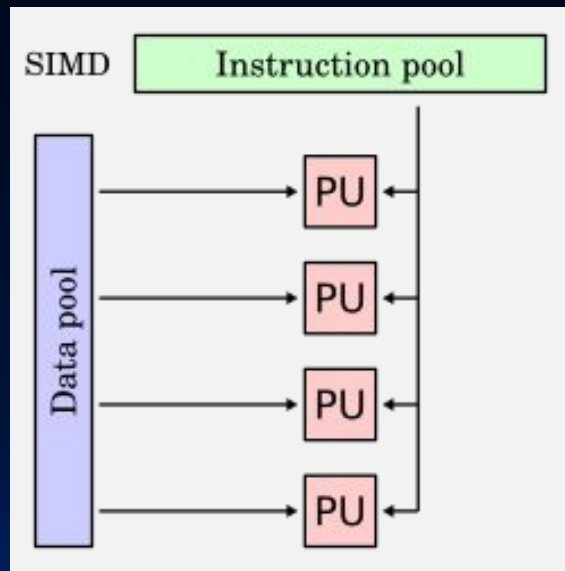
SISD: Single Instruction/Single Data Stream

- Serial computer that exploits **no parallelism** in either the instruction or data streams
- Examples: Traditional uniprocessor machines
 - Our RISC-V processor
 - Superscalar (because the **programming model** is sequential)
- SISD** is what we did up to now in CS61C.



SIMD: Single Instruction/Multiple Data Stream

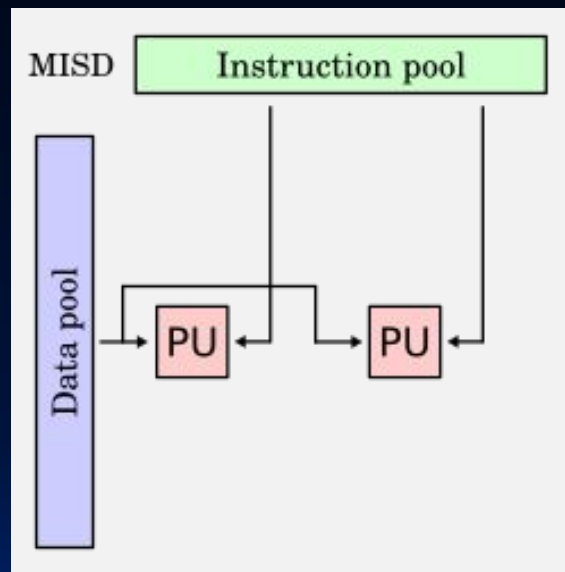
- **SIMD**: "sim-dee"
- Computer that applies a single instruction stream to **multiple data streams** for operations that may be naturally parallelized
- Examples:
 - Intel SIMD instruction extensions
 - NVIDIA Graphics Processing Unit (GPU)



NVIDIA GTX 40
GPU Series

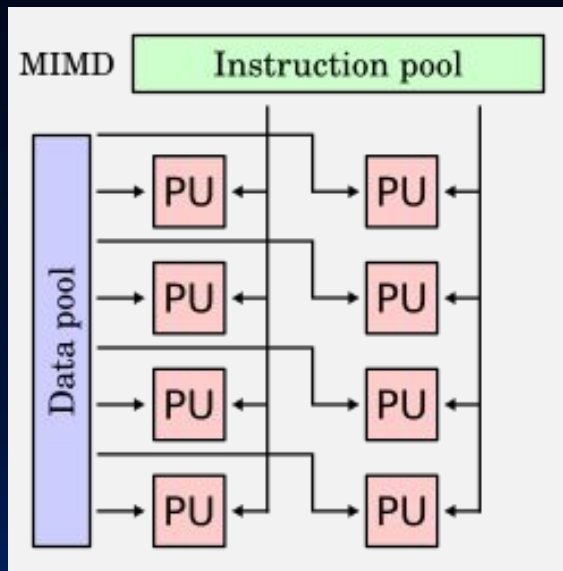
MISD: Multiple Instruction/Single Data Stream

- Exploits multiple instruction streams against a **single data stream** for data operations that can be naturally parallelized
- Examples:
 - None nowadays...
 - Historical significance, e.g., certain kinds of array processors



MIMD: Multiple Instruction/Multiple Data Stream

- **MIMD**: “mim-dee” (more next week)
- Multiple autonomous processors simultaneously executing **different instructions on different data**
- Examples:
 - Multicore
 - Warehouse Scale Computers
 - “Single program, multiple data” (one program runs on all processors)



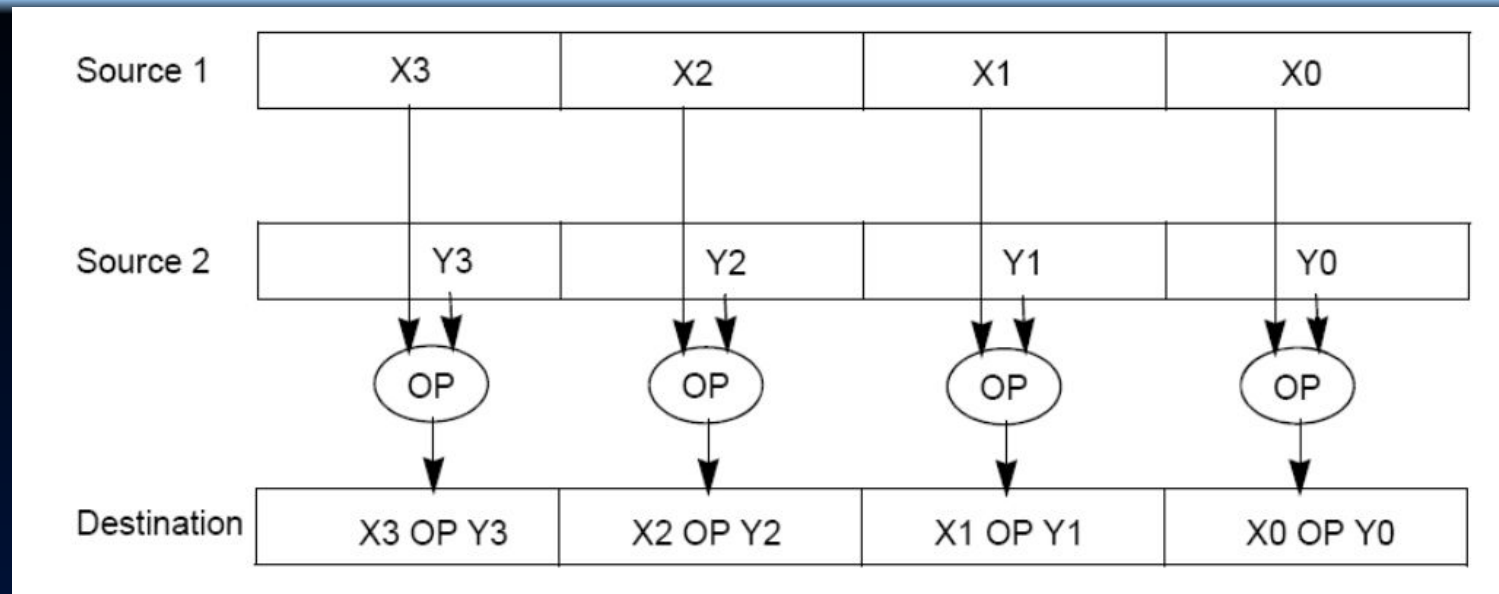
Apple M4
[Tech Insights]



SIMD and Data-Level Parallelism

- Dgemm, continued
- Flynn's Taxonomy
SIMD and Data-Level
Parallelism
- SIMD dgem
Intel Intrinsics

Fetch one instruction, do the work of multiple instructions



- Source operands + destination **registers wide enough** to fit multiple values (e.g., four 64-bit floating point numbers)
- Apply single operation (e.g., OP:= add) to all operands in register



SIMD is Data-Level Parallelism

- **Data-Level Parallelism (DLP):** Executing one operation on multiple data streams
- Example: Element-wise vector multiplication

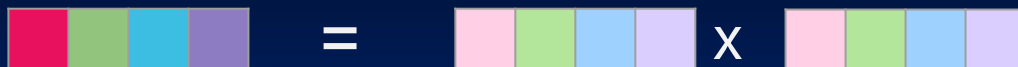
Python `y = [c * x for c, x in zip(cs, xs)]`

C
`for (int i = 0; i < n; i++) {
 y[i] = c[i] * x[i]
}`

Snap!

numbers from 1 to 4 × numbers from 1 to 4

1	1
2	4
3	9
4	16
length: 4	





SIMD Applications & Implementations

Applications

- Scientific computing (Matlab, NumPy)
- Graphics and video processing (Photoshop, ...)
- Big Data (Deep learning)
- Gaming

Implementations

- x86 **Intel Intrinsics**
- ARM
- RISC-V vector extensions
 - (see CS152)
- Video cards

SIMD instructions can often be accessed via **extensions** to a given ISA, e.g., Intel x86.



Agenda

SIMD dgemm

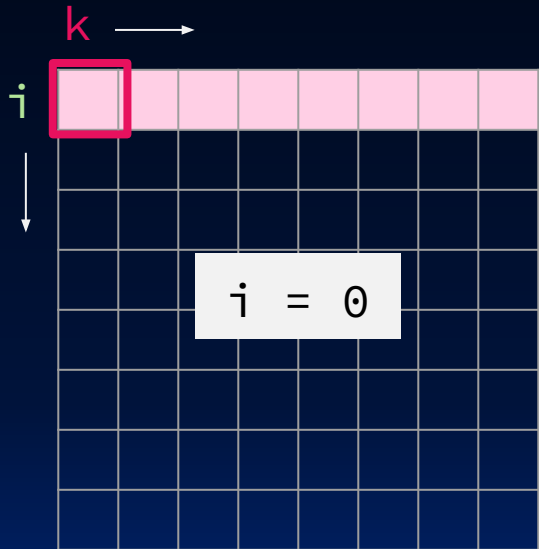
- Dgemm, continued
- Flynn's Taxonomy
SIMD and Data-Level Parallelism
- SIMD dgem
Intel Intrinsics

SISD: Scalar dgemm

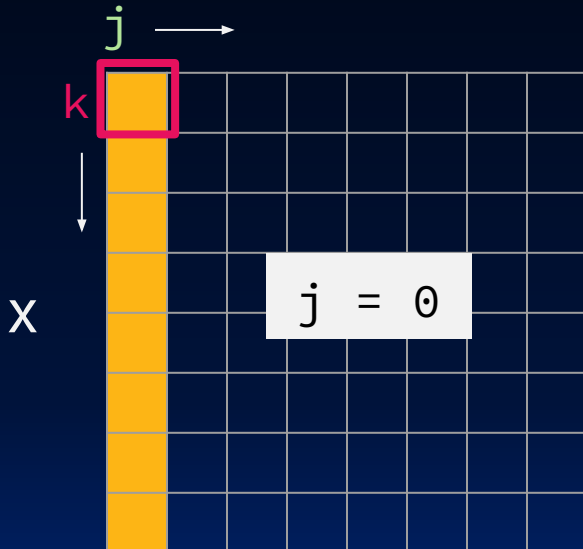
Iterate through **k** to create the element in C.

- "columns" of i-th row of A
- "rows" of j-th col of B

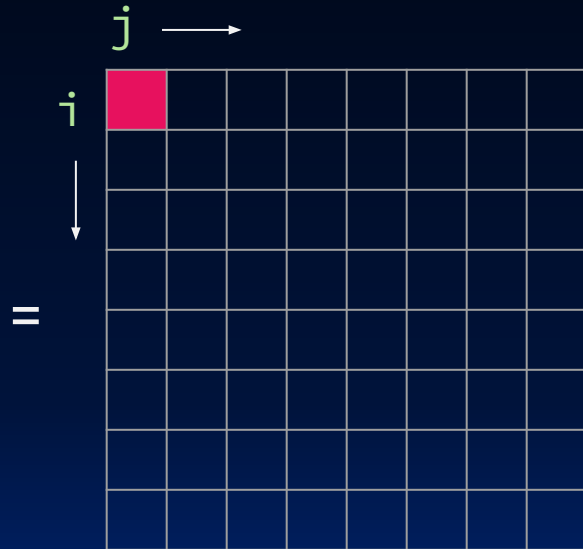
```
double Cij = 0; // i = 0, j = 0
for (int k = 0; k < N; k++) {
    Cij += A[i+k*N] * B[k+j*N];
}
```



A



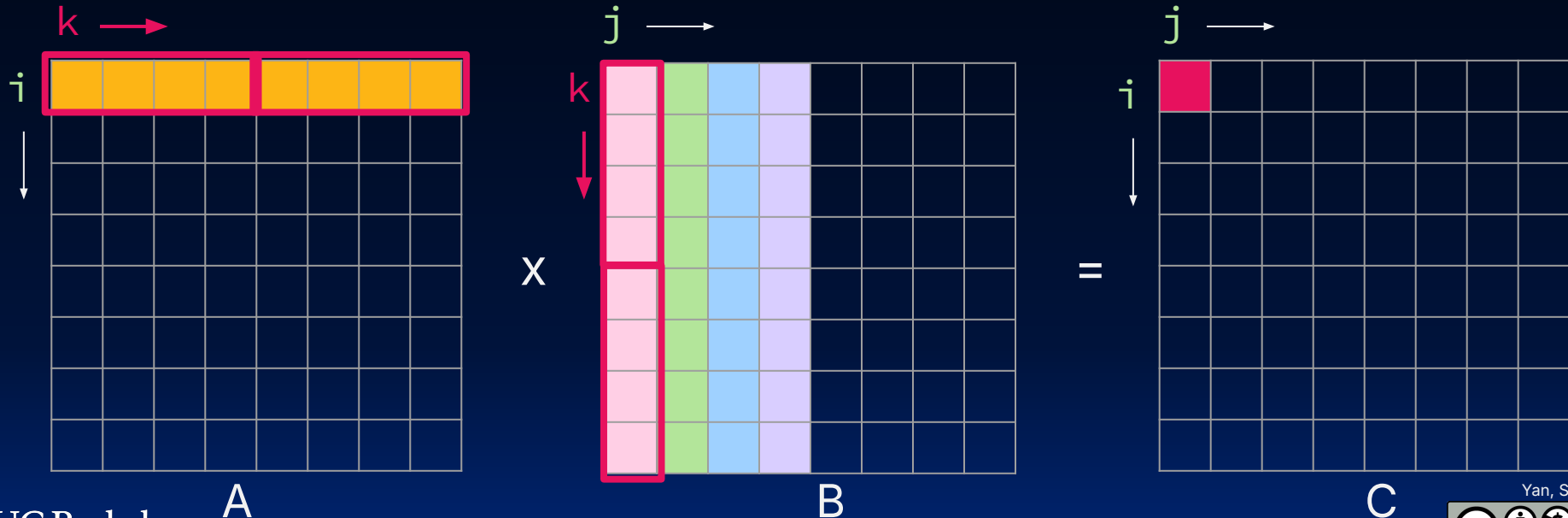
B



C

Naive SIMD dgemm

- Unfortunately, simple strides don't resolve our **cache misses** from loading in rows of B instead of columns.
- Could transpose...





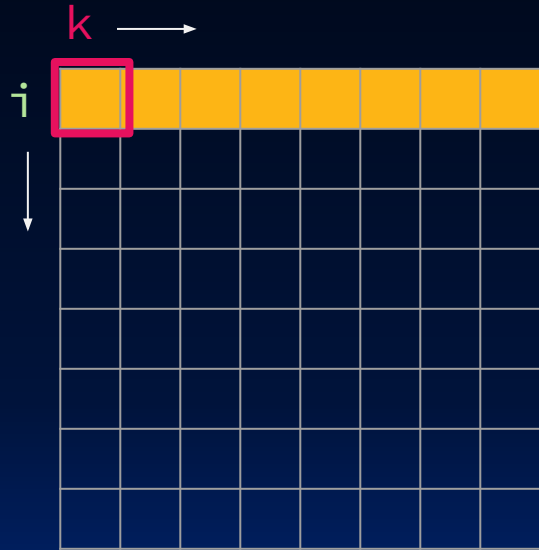
SIMD dgemm with four-double vector

- How can we utilize **cache blocking** with SIMD?
- Our goal for the innermost loop (over k):

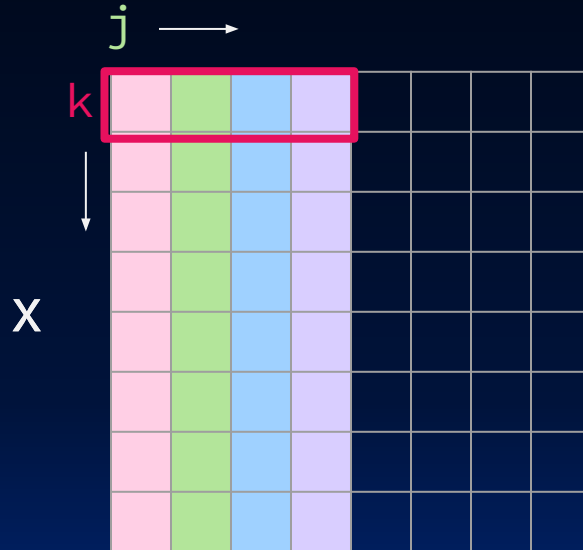


SIMD dgemm with four-double vector (1a/4)

```
// i = 0, j = 0
```

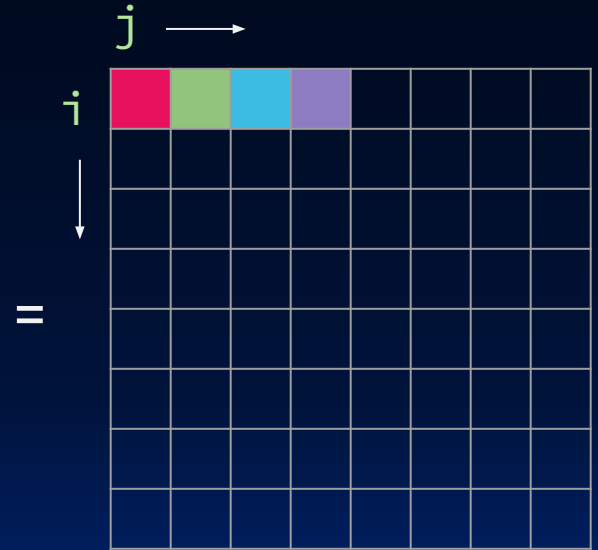


A



X

B



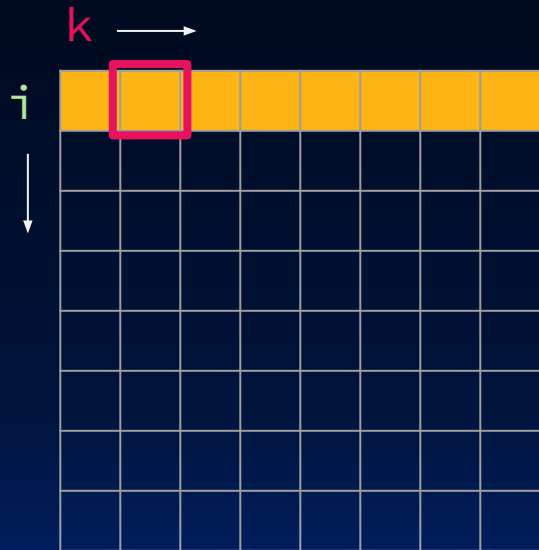
=

C

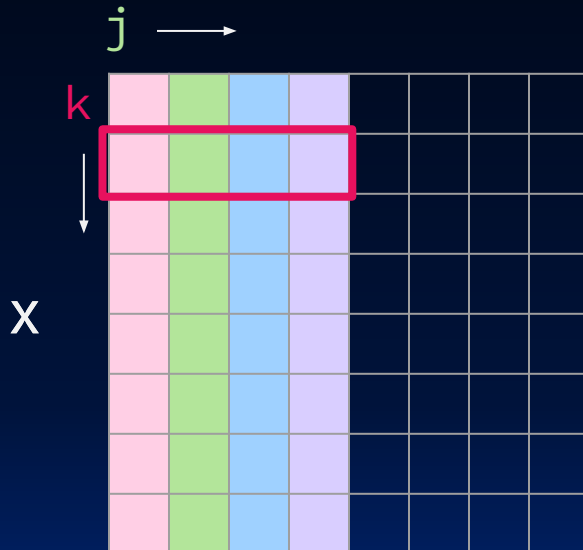


SIMD dgemm with four-double vector (1b/4)

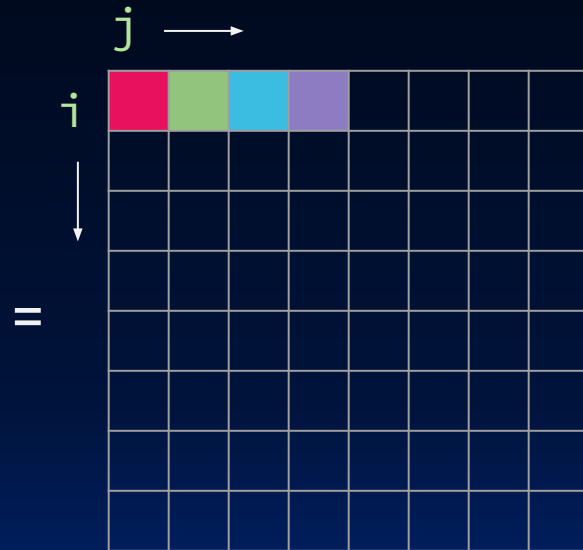
// **i = 0**, j = 0



A



B



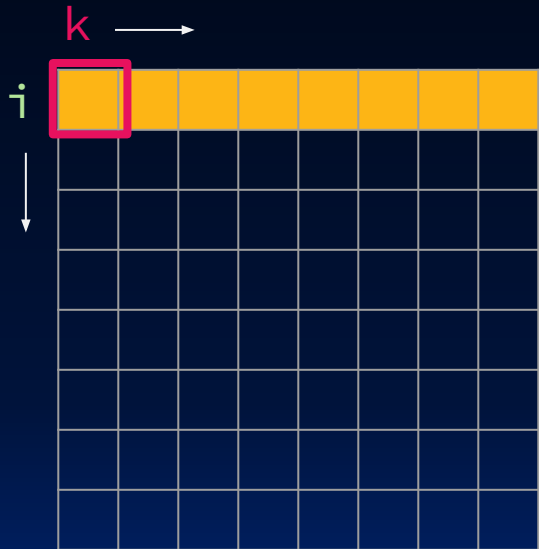
C

X

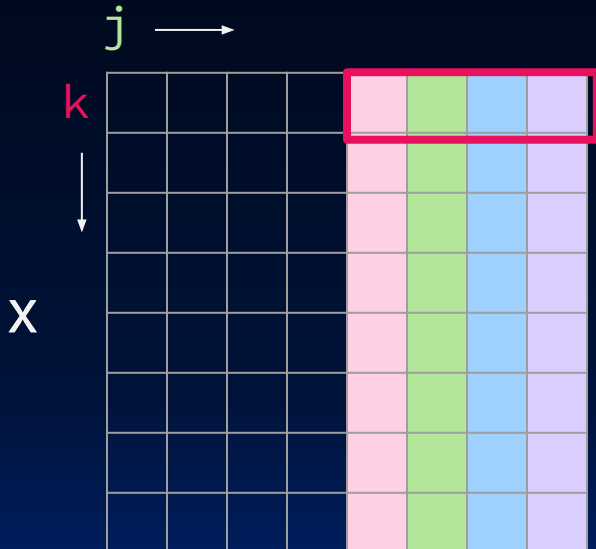
=

SIMD dgemm with four-double vector (2/4)

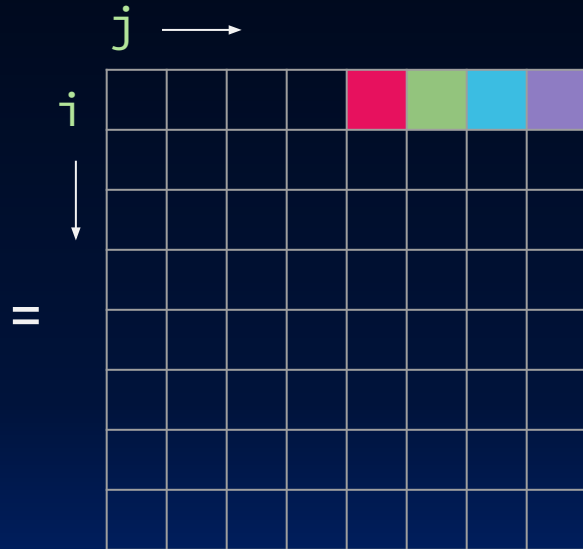
// $i = 0$, $j = 4$



A



B



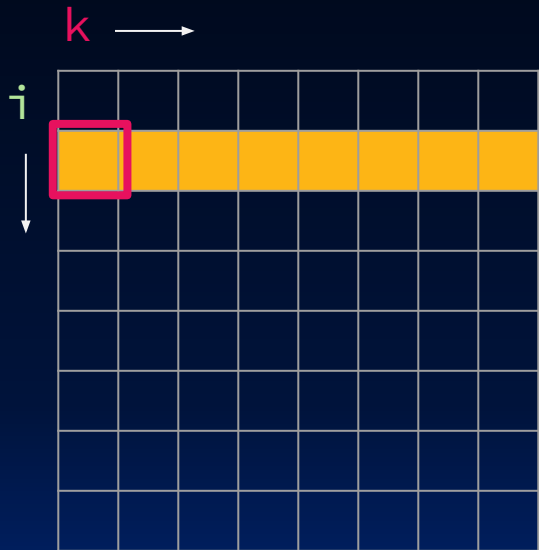
C

X

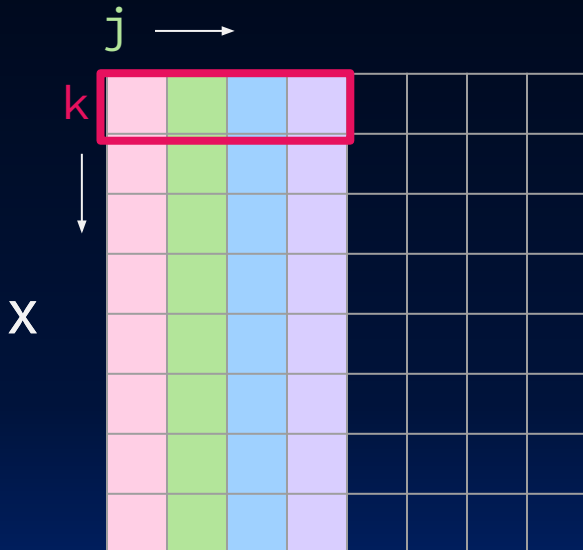
=

SIMD dgemm with four-double vector (3/4)

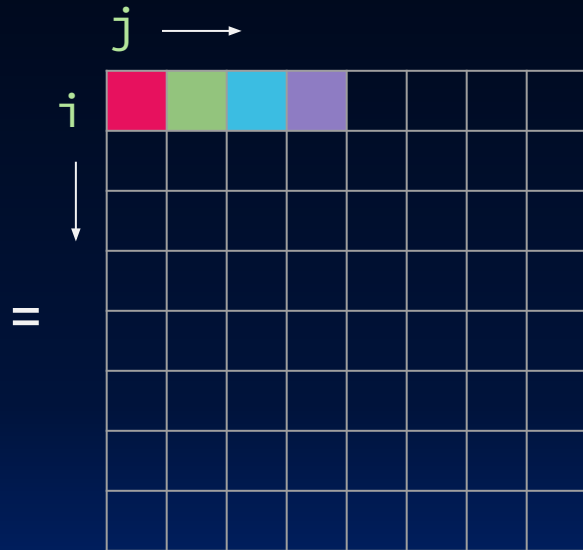
// **$i = \underline{1}$** , $j = 0$



A



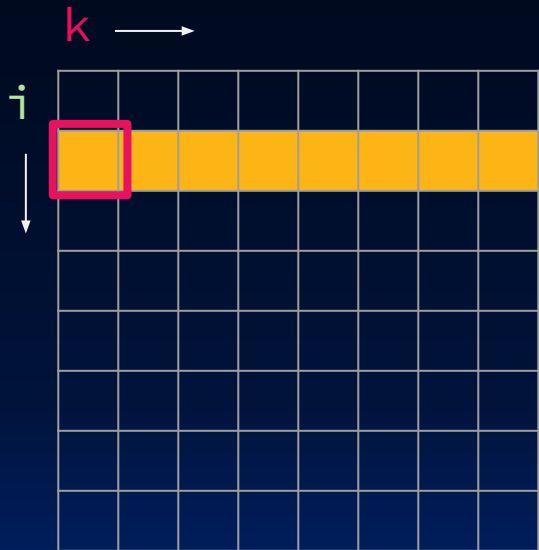
B



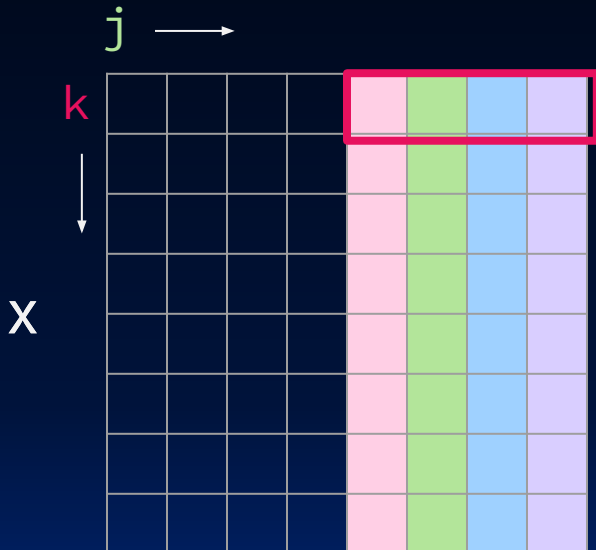
C

SIMD dgemm with four-double vector (4/4)

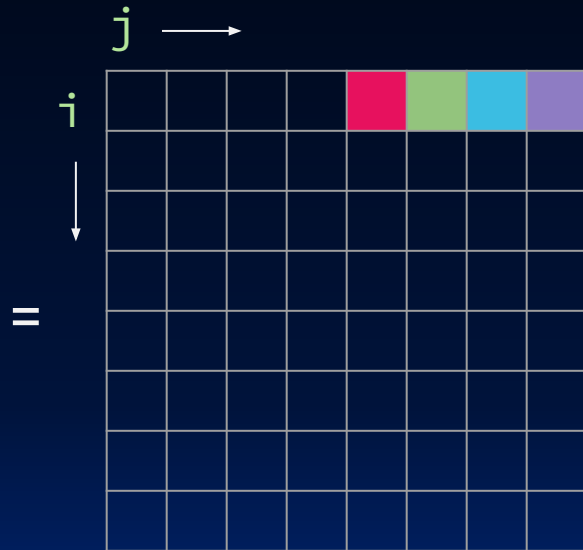
// $i = 1$, $j = 4$



A



B



C

X

=



dgemm benchmark, SIMD

- SIMD **4x faster** than scalar!
- ...but still way lower than the peak 25 GFLOPs performance on this CPU...?
- Possible culprits:
 - Cache
 - Pipeline hazards
 - See CS61C Fall 2021 Lecture 21 for more details

	GFLOPS		
N	Python	C (scalar)	C + AVX
32	0.0054	1.30	4.56
160	0.0055	1.30	5.47
480	0.0054	1.32	5.27
960	0.0053	0.91	3.64

Characteristic	Value
CPU	i7-5557U
Clock rate (sustained)	3.1 GHz
Instructions per clock (mul_pd)	2
Parallel multiplies per instruction	4
Peak double FLOPS	24.8 GFLOPS



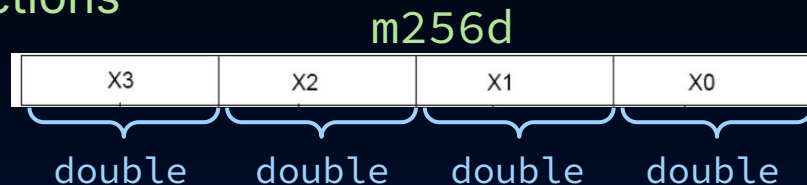
Intel Intrinsics

- Dgemm, continued
- Flynn's Taxonomy
SIMD and Data-Level Parallelism
- SIMD dgem
Intel Intrinsics



SIMD Instructions as Assembly Extensions

- **SIMD instructions** can often be accessed via **extensions** to a given ISA, e.g., Intel x86.
 - Sometimes known as vector instructions
- Use specialized "vector" registers
 - Wide enough to fit multiple values



In this class, we will write SIMD code in C using Intel intrinsics, instead of writing direct assembly code.

Intel x86 SIMD Evolution (1/2)

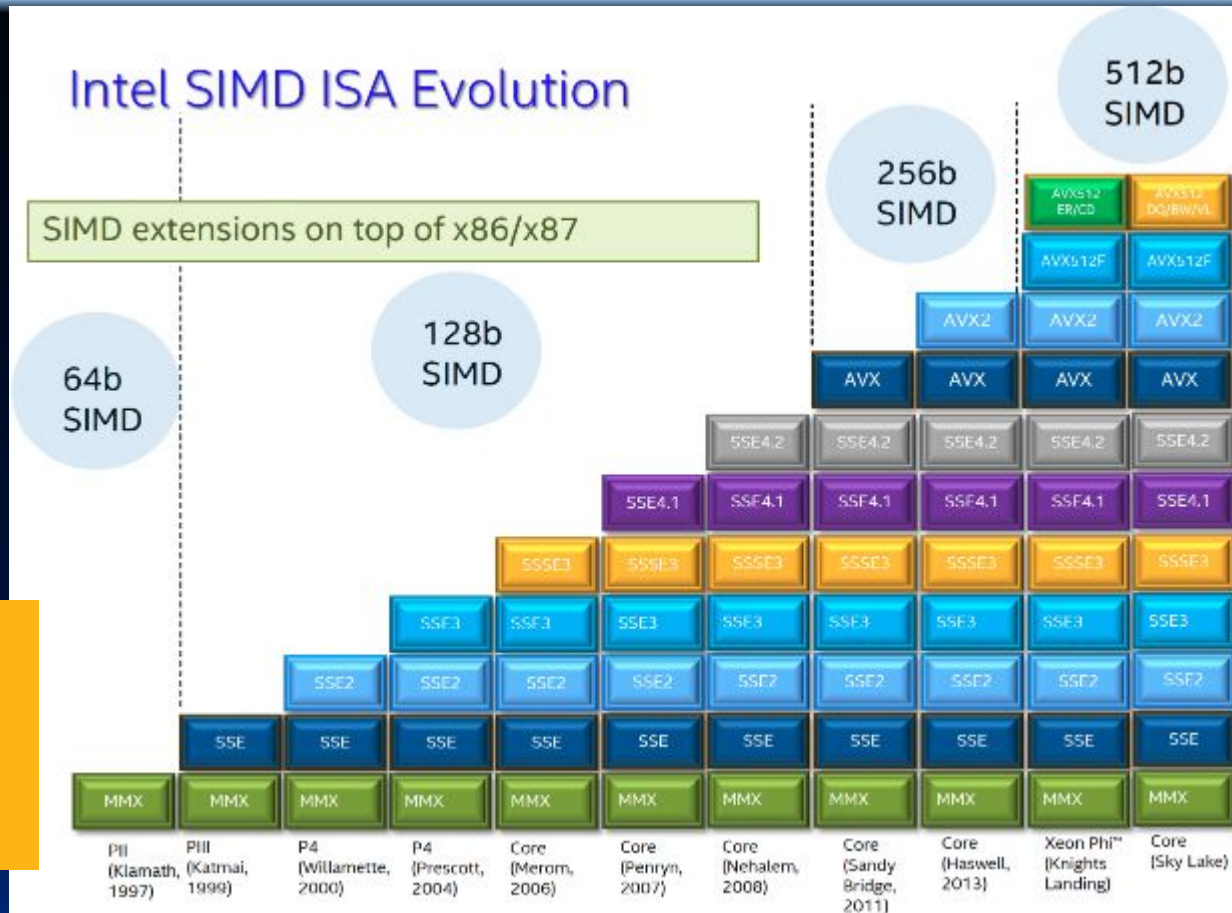
Naming of SIMD extensions has changed with functionality:

- New instructions every few years
- New and wider registers
- More parallelism

Intel architectures are generally all backwards compatible! (but docs get very long)

Intel SIMD ISA Evolution

SIMD extensions on top of x86/x87





Intel x86 SIMD Evolution: Intrinsics

- **Intrinsics** are C functions and procedures that provide access to assembly language.
 - With intrinsics, can program using assembly instructions indirectly.
 - One-to-one correspondence between SIMD extension assembly instruction (e.g., SSE, AVX) and intrinsics.

[Intel docs](#)

In this class, we will write SIMD code in C using Intel intrinsics, instead of writing direct assembly code.



Intel Intrinsics: Instructions and Format

[Intel docs](#)

`_mm<register width>_<instruction>_<data type>`

`__m256i _mm256_add_epi32 (__m256i a, __m256i b);`

256-bit register add 32-bit integer



`__m128 _mm_load_ps (float const* mem_addr);`

128-bit register load 32-bit float



Load 128-bits (of 4 packed single-precision 32-bit floats) from memory into dst. mem_addr must be aligned on a 16-byte boundary.